

```
1 # !pip install tensorflow
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.9.23)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.0)
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.20.3)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.2.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.76.0)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.10.0)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.15.1)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.3)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.45.1)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2025.10.5)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (3.9)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (3.1.3)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboard~=2.19.0->tensorflow) (3.0.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->tensorflow) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->tensorflow) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow) (0.1.2)
```

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
1 import tensorflow as tf
2 import numpy as np
3 from PIL import Image
4 import os
```

```
1 import tensorflow as tf
2 import os
3
4 # --- Updated paths based on your files ---
5 xception_path = '/content/drive/MyDrive/deepfake_pretrained_model_files/140K_resnet50_model.keras'
6 resnet_path = '/content/drive/MyDrive/deepfake_pretrained_model_files/140K_xception_model.keras'
7 efficientnet_path = '/content/drive/MyDrive/deepfake_pretrained_model_files/deepfake (1).h5'
8 # -----
9
10 try:
11     xception_model = tf.keras.models.load_model(xception_path)
12     resnet_model = tf.keras.models.load_model(resnet_path)
13     efficientnet_model = tf.keras.models.load_model(efficientnet_path)
14
15     models = {
16         'xception': xception_model,
17         'resnet50': resnet_model,
18         'efficientnet': efficientnet_model
19     }
20     print("All models loaded successfully!")
21
22 except Exception as e:
23     print(f"Error loading models: {e}")
24     print("Please ensure the model file paths are correct and the files exist.")
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.
All models loaded successfully!

```
1 def preprocess_image_for_model(img_pil, model_name):
2     """Applies the correct preprocessing for a given model."""
3
4     if model_name == 'xception':
5         # Target size: (256, 256), Scaling: 1.0/255
6         img_resized = img_pil.resize((256, 256))
7         img_array = tf.keras.preprocessing.image.img_to_array(img_resized)
8         img_array /= 255.0
9         return np.expand_dims(img_array, axis=0)
10
11     elif model_name == 'resnet50':
12         # Target size: (256, 256), Scaling: 1.0/255
13         img_resized = img_pil.resize((256, 256))
14         img_array = tf.keras.preprocessing.image.img_to_array(img_resized)
15         img_array /= 255.0
16         return np.expand_dims(img_array, axis=0)
17
18     elif model_name == 'efficientnet':
19         # Target size: (224, 224), Scaling: None (Internal to model)
20         img_resized = img_pil.resize((224, 224))
21         img_array = tf.keras.preprocessing.image.img_to_array(img_resized)
22         # No rescaling needed as it's handled inside the model
23         return np.expand_dims(img_array, axis=0)
24     else:
25         raise ValueError("Unknown model name")
```

```
1 import numpy as np
2 import os
3 from PIL import Image
4
5 # Based on our analysis: 'Fake' is 0, 'Real' is 1.
6 # A sigmoid output gives the probability of the *positive* class (class 1), which is 'Real'.
7
8 def ensemble_predict(img_path):
9     """
```

```
10     Runs prediction on all models and returns the ensemble result.
11     Returns the final label, average probability of being 'Real', and individual scores.
12     """
13     if not os.path.exists(img_path):
14         return f"Error: Image path not found at '{img_path}'", None, None
15
16     try:
17         img_pil = Image.open(img_path).convert('RGB')
18     except Exception as e:
19         return f"Error opening image: {e}", None, None
20
21     all_probs = {}
22
23     print("Running predictions...")
24     for name, model in models.items():
25         # 1. Preprocess the image specifically for this model
26         processed_img = preprocess_image_for_model(img_pil, name)
27
28         # 2. Get the prediction
29         pred = model.predict(processed_img)[0]
30
31         # 3. Standardize the output probability for the 'Real' class
32         # ResNet50 & Xception (binary) output is a single value like [0.98]
33         # EfficientNet (categorical) output is like [0.02, 0.98] for (Fake, Real)
34         prob_real = pred[1] if len(pred) > 1 else pred[0]
35
36         all_probs[name] = float(prob_real)
37
38     # 4. Average the probabilities (soft voting) for the 'Real' class
39     avg_prob_real = np.mean(list(all_probs.values()))
40
41     # 5. Determine the final class based on the average probability
42     final_prediction = "Real" if avg_prob_real >= 0.5 else "Fake"
43
44     return final_prediction, avg_prob_real, all_probs
```

```
1 from google.colab import files
2 import matplotlib.pyplot as plt
3 import matplotlib.image as mpimg
4
5 uploaded = files.upload()
6
7 # Get the filename of the uploaded file
8 if len(uploaded.keys()) > 0:
9     test_image_path = list(uploaded.keys())[0]
10    print(f"\nImage '{test_image_path}' uploaded successfully!")
11
12    # Optional: Display the uploaded image
13    img = mpimg.imread(test_image_path)
14    plt.imshow(img)
15    plt.axis('off')
16    plt.show()
17 else:
18     test_image_path = None
19     print("No file was uploaded.")
```

Choose files fake_10010.jpg

fake_10010.jpg(image/jpeg) - 9053 bytes, last modified: 06/11/2025 - 100% done
Saving fake_10010.jpg to fake_10010.jpg

Image 'fake_10010.jpg' uploaded successfully!



```
1 if test_image_path:
2     final_label, avg_prob, individual_scores = ensemble_predict(test_image_path)
3
4     if final_label and "Error" not in final_label:
5         print("\n--- Ensemble Results ---")
6         print(f"Final Prediction: {final_label}")
7         print(f"Average Confidence (for 'Real'): {avg_prob:.4f}")
8         print("\n--- Individual Model Scores (for 'Real') ---")
9         for model_name, score in individual_scores.items():
10             print(f"{model_name:>15}: {score:.4f}")
11     else:
12         print(final_label) # This will print an error message if something went wrong.
13 else:
14     print("Please upload an image in the cell above before running the prediction.")
```

Running predictions...

1/1 ██████████ 0s 45ms/step
1/1 ██████████ 0s 44ms/step
1/1 ██████████ 0s 35ms/step

--- Ensemble Results ---
Final Prediction: Fake
Average Confidence (for 'Real'): 0.4124

--- Individual Model Scores (for 'Real') ---
xception: 0.0001
resnet50: 0.9965
efficientnet: 0.2405

```
1 !pip install -q gradio
2 import tensorflow as tf
3 import numpy as np
4 from PIL import Image
```

```

1 def predict_deepfake(pil_image):
2
3     if pil_image is None:
4         return None, "Please upload an image."
5
6     all_probs = {}
7
8     # Ensure image is in RGB format
9     pil_image = pil_image.convert('RGB')
10
11    for name, model in models.items():
12        processed_img = preprocess_image_for_model(pil_image, name)
13        pred = model.predict(processed_img, verbose=0)[0]
14
15        # Standardize the output probability for the 'Real' class
16        prob_real = pred[1] if len(pred) > 1 else pred[0]
17        all_probs[name] = float(prob_real)
18
19    # Calculate average probability for 'Real'
20    avg_prob_real = np.mean(list(all_probs.values()))
21
22    # Create the dictionary for the Gradio Label component
23    # It shows probabilities for both classes
24    confidences = {'Real': avg_prob_real, 'Fake': 1 - avg_prob_real}
25
26    # Create a formatted string for the individual scores
27    individual_scores_str = ""
28    for model_name, score in all_probs.items():
29        individual_scores_str += f"{model_name:>15}: {score:.4f} (Prob. for 'Real')\n"
30
31    return confidences, individual_scores_str

```

```

16 # Xception Architecture (loaded into resnet_model variable in your notebook)
17 'resnet50': 'block14_sepconv2_act',
18
19 # EfficientNetB0 Base (loaded into efficientnet_model variable)
20 'efficientnet': 'top_conv'
21 }
22
23
24 def make_gradcam_heatmap(img_array, model, model_name, last_conv_layer_name, pred_index):
25     """Generates Grad-CAM heatmap for a given model and target layer."""
26
27     # Handle EfficientNet sub-model access
28     if model_name == 'efficientnet':
29         base_model = model.get_layer('efficientnetb0')
30         # We create a new model from the base model's input to the desired layer
31         grad_model = tf.keras.models.Model(
32             base_model.inputs, [base_model.get_layer(last_conv_layer_name).output, model.output]
33         )
34     else:
35         # Standard model access
36         grad_model = tf.keras.models.Model(
37             model.inputs, [model.get_layer(last_conv_layer_name).output, model.output]
38         )
39
40     # Compute gradient
41     with tf.GradientTape() as tape:
42         last_conv_layer_output, preds = grad_model(img_array)
43         class_channel = preds[:, pred_index]
44
45     # Gradient of the output neuron (wrong class) w.r.t the activation map
46     grads = tape.gradient(class_channel, last_conv_layer_output)
47
48     # Global average pooling of the gradients to get weights
49     pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
50
51     # Multiply each channel by its weight and sum up to get the heatmap
52     last_conv_layer_output = last_conv_layer_output[0]
53     heatmap = last_conv_layer_output @ pooled_grads[..., tf.newaxis]
54     heatmap = tf.squeeze(heatmap)
55
56     # Normalize the heatmap (ReLU to only keep positive gradients)
57     heatmap = tf.maximum(heatmap, 0) / (tf.reduce_max(heatmap) + 1e-10) # Added epsilon to prevent division by zero
58
59     return heatmap.numpy()
60
61
62 def visualize_gradcam(original_img_pil, heatmap, model_name):
63     """Overlays the heatmap onto the original image and displays it."""
64
65     # Convert PIL image to CV2 format (BGR)
66     orig_img_cv2 = np.array(original_img_pil.convert('RGB'))
67     orig_img_cv2 = cv2.cvtColor(orig_img_cv2, cv2.COLOR_RGB2BGR)
68
69     # Rescale heatmap to match original image size (using PIL for simplicity in resizing)
70     heatmap = Image.fromarray(np.uint8(255 * heatmap))
71     heatmap = heatmap.resize(original_img_pil.size)
72     heatmap = np.array(heatmap)
73
74     # Apply JET colormap to the heatmap
75     heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
76
77     # Overlay the heatmap on the original image
78     # Note: cv2.addWeighted requires images of the same size and type
79     superimposed_img = cv2.addWeighted(orig_img_cv2, 0.6, heatmap, 0.4, 0)
80
81     # Convert back to PIL/RGB for Matplotlib display
82     superimposed_img_rgb = cv2.cvtColor(superimposed_img, cv2.COLOR_BGR2RGB)
83
84     plt.figure(figsize=(6, 6))
85     plt.imshow(superimposed_img_rgb)
86     plt.title(f"Grad-CAM on {model_name} (Attribution for Wrong Class)")
87     plt.axis('off')
88     plt.show()
89
90 # You should also import these libraries:
91 # import cv2

```

```

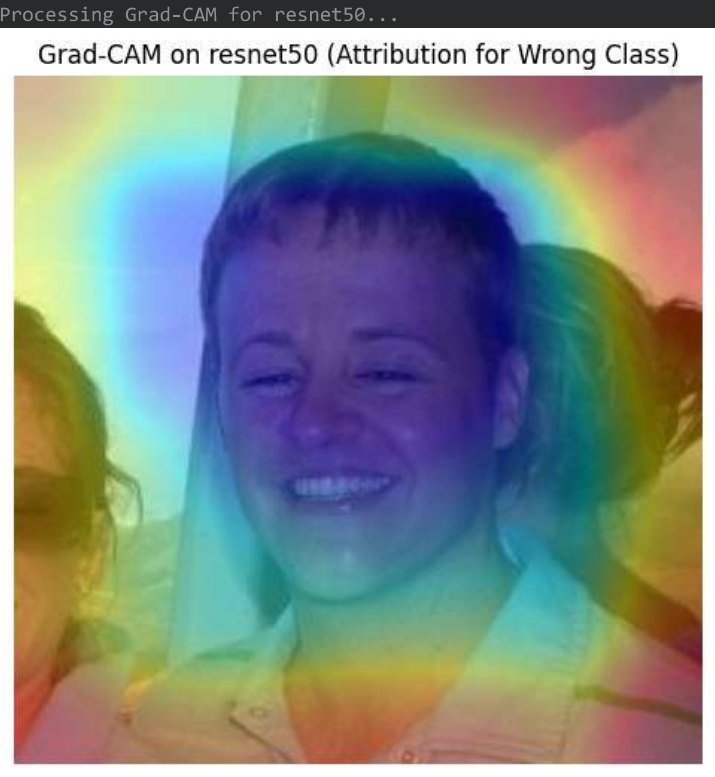
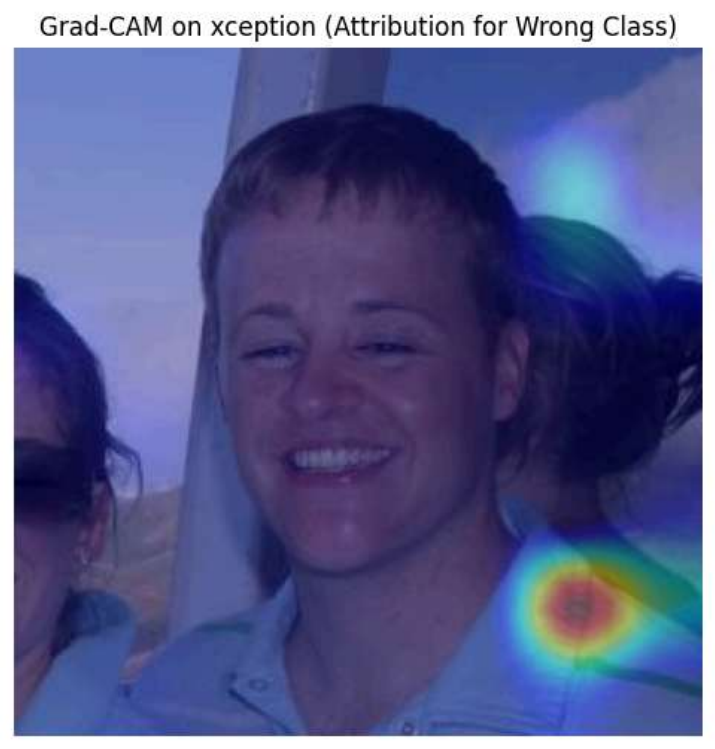
1 # -----
2 # *** MANUAL STEP: SET THE GROUND TRUTH FOR YOUR TEST IMAGE ***
3 # Set to True if the image is actually REAL, False if it is actually FAKE.
4 # -----
5 ground_truth_is_real = True # <--- CHANGE THIS IF YOUR IMAGE IS FAKE
6
7 # --- Logic to check for misclassification and run Grad-CAM ---
8
9 if 'final_label' in locals() and final_label:
10     predicted_is_real = final_label == "Real"
11
12     if predicted_is_real != ground_truth_is_real:
13         print("\n🔥 MISCLASSIFICATION DETECTED! Running Grad-CAM for all models...")
14
15         try:
16             # Load the original PIL image
17             img_pil = Image.open(test_image_path).convert('RGB')
18         except Exception as e:
19             print(f"Error loading image for Grad-CAM: {e}")
20             exit()
21
22         # Determine the target index (the *wrongly* predicted class index)
23         # Class 0 = Fake, Class 1 = Real
24         target_class_index = 0 if ground_truth_is_real else 1
25
26         for name, model in models.items():
27             print(f"\nProcessing Grad-CAM for {name}...")
28
29             # 1. Preprocess image for the current model
30             processed_img = preprocess_image_for_model(img_pil, name)
31
32             # 2. Generate heatmap for the WRONG class
33             heatmap = make_gradcam_heatmap(
34                 processed_img,
35                 model,
36                 name, # Pass model name to handle sub-model access
37                 last_conv_layers[name],

```

```
38         pred_index=target_class_index
39     )
40
41     # 3. Visualize
42     visualize_gradcam(img_pil, heatmap, name)
43 else:
44     print("\n✅ Correctly classified. Grad-CAM not triggered.")
45
46 else:
```

🔥 MISCLASSIFICATION DETECTED! Running Grad-CAM for all models...

Processing Grad-CAM for xception...
/usr/local/lib/python3.12/dist-packages/keras/src/models/functional.py:241: UserWarning: The structure of `inputs` doesn't match the expected structure.
Expected: ['input_layer']
Received: inputs=Tensor(shape=(1, 256, 256, 3))
warnings.warn(msg)



Processing Grad-CAM for efficientnet...

```
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipython-input-279867289.py in <cell line: 0>()
    31
    32     # 2. Generate heatmap for the WRONG class
--> 33     heatmap = make_gradcam_heatmap(
    34         processed_img,
    35         model,
```

⬆ 2 frames

```
/usr/local/lib/python3.12/dist-packages/keras/src/ops/operation.py in _get_node_attribute_at_index(self, node_index, attr, attr_name)
    305     """
    306     if not self._inbound_nodes:
--> 307         raise AttributeError(
    308             f"The layer {self.name} has never been called "
    309             f"and thus has no defined {attr_name}."
```

AttributeError: The layer sequential has never been called and thus has no defined output.

Next steps: [Explain error](#)

1 Start coding or [generate](#) with AI.

